Day 14 (Nov 2): Retest and Python Dict Fix Session

Git and Excel retest | Lists, tuples, dicts, sets recall | one engine for dict problems. Fresh from memory, no notes.

1. KEY IDEAS

Python: the dict engine (took longer, more depth)

- Every dict problem has the same parts: **1) where the key comes from** (part of the item like name or word[0], or a rule like a condition), **2) the first-time branch** (key missing: store the first value, or start at 0), **3) the update** (every time after).
- Three kinds of update: **add 1** (count), **add the value** (total), **compare and replace** (best or longest). Compare needs a tie rule.
- **Ties**: with > the first one stays; with >= the last one wins. Say which the task wants. **Boundaries**: "15 and above" means 15 is in the top bucket; test 9, 10, 14, 15.
- If the keys are known in advance, **pre-seed** them with 0 and use += 1. If they are not known (words in a sentence), use get(key, 0) + 1. A += on a missing key raises KeyError.
- A correct output can be a coincidence. Always test a second input with the **order reversed** (or a boundary, or an empty input).
- Never name a variable dict, list or max: it hides Python's own name.

Git and Excel retest

- Different files on both branches = merge commit (diverged). Same line = conflict. No new commit on main = fast-forward. **--ff-only** refuses anything but a fast-forward; **--no-ff** forces a merge commit.
- IFERROR catches #N/A, #DIV/0!, #REF!, #VALUE!, #NAME?, #NUM!, #NULL!; IFNA catches only #N/A, so real mistakes still show.

2. EXAMPLES TO KNOW

```
best = {}
for name, mark in records:        # records = [("Asha", 80), ("Ravi", 65), ("Asha", 90), ...]
    if name not in best:          # key missing: store the first mark
        best[name] = mark
    elif mark > best[name]:       # every time after: replace only if higher
        best[name] = mark
# {'Asha': 90, 'Ravi': 70, 'Meena': 92}
```

```
longest = {}
for word in words:                # ["apple", "bat", "ant", "bird", "cat", "bear"]
    key = word[0]
    if key not in longest:
        longest[key] = word
    elif len(word) > len(longest[key]):   # > keeps the first word on a tie
        longest[key] = word
# {'a': 'apple', 'b': 'bird', 'c': 'cat'}
```

```
result = {'small': 0, 'medium': 0, 'large': 0}
for n in nums:                    # 1 to 9 small, 10 to 14 medium, 15 and above large
    if n >= 1 and n <= 9:
        result['small'] += 1
    elif n >= 10 and n <= 14:
        result['medium'] += 1
    else:
        result['large'] += 1
# [3, 8, 12, 7, 20, 5, 15] -> {'small': 4, 'medium': 1, 'large': 2}
```

3. MY MISTAKES AND THE FIX

What I said or did	The fix
if v not in dict: (checked the score)	Check the key: if name not in best. A number is never a key, so the elif never ran.
elif dict[n] > v: (kept the lower mark)	To keep the best, replace when mark > best[name]. Two bugs hid each other.
Output was right, so the code was right	It matched by luck. A reversed-order input gave 80 for a best of 90.
'A string is greater than any number'	In Python 3, str > int raises TypeError: '>' not supported between instances of 'str' and 'int'.
'Same letter, so bird is not considered'	Same first letter put them on one key. Bird was compared: 4 vs 4 is not greater, so bear stayed.
>= keeps bird on a tie	> keeps the first one; >= lets the last one win.
15 falls in medium	'15 and above' is large. Test 9, 10, 14, 15.
else catches the rest (0 and negatives became large)	Give large its own condition (n >= 15); decide what the else means.
KeyError 'prints'; named a variable dict	An error is raised, not printed. dict is Python's own name.

4. HOW I IMPROVED

- Recalls on Lists and tuples and Dicts and sets: all six answers correct, with full dict traces after every round.
- Found and fixed the double bug myself after one nudge, then solved the longest-word problem without help.
- Git retest: all three merge cases correct, with the cause of each.

5. PRACTICE TO RETRY (no solutions)

- Trace counts.get on "to be or not to be": write the whole dict after every round.
- Topper with a tie on 92: print **all** toppers. Then say what changes with >=.
- Marks list [49, 50, 74, 75, 100]: count bands (below 50 fail, 50 to 74 pass, 75 and above distinction); predict 49, 50, 74, 75 first.
- First letter -> shortest word, with ties keeping the last. Say which comparison you need before coding.

6. RECALL QUESTIONS (answer from memory, no notes)

- What are the three parts of every dict problem? Name three kinds of update.
- Why was the if v not in dict version wrong, and why did its output still look right?
- When do you pre-seed keys with 0, and when do you use get()?
- What does > do on a tie, and what does >= do?

7. OPEN ITEMS

- Python retake on a new problem (Nov 3 afternoon). Functions lesson and is_prime moved to Nov 4.
- Business topic, AI Fluency, AI skill hour; clone and conflict practice on Nov 3.